

CityTech TTP Summer Bootcamp

CSS, State, and the Mini Project

2026-06-10

Agenda

1. Review
2. Follow-Up: Library vs Framework
3. CSS
4. React State
5. Mini Project

Review

What are the most important things you remember from yesterday?

Follow Up: Library vs Framework

- Both are packages of code written by a 3rd party
- A **library** is code *you* call — you stay in control of the flow. Generally **easy to swap out**.
- A **framework** dictates the structure and lifecycle. A **long-term commitment** that's harder to replace later.

Intro to CSS

CSS (Cascading Style Sheets) controls how your HTML **looks** — colors, fonts, spacing, layout.

```
h1 {  
  color: navy;  
  font-size: 2rem;  
}
```

- **Selectors** target elements; **properties** set their styles
- HTML is the structure; CSS is the presentation

CSS Selectors

Selectors pick **which elements** a rule styles:

- **Type** — by tag name: `p`, `h1`
- **Class** — by `class="..."`: `.btn` (reusable, most common)
- **ID** — by `id="..."`: `#header` (unique)
- **Descendant** — nested elements: `nav a`

```
.btn    { color: white; }  
#header { height: 60px; }  
nav a   { text-decoration: none; }
```

Overriding CSS (the Cascade)

When multiple rules target the same element, CSS decides which one wins — that's the "Cascading" in CSS.

- **Specificity** — more specific selectors win: ID (`#id`) > class (`.class`) > type (`p`)
- **Source order** — on a tie, the rule written **last** wins
- **Inline styles** and **!important** override the rest (use sparingly)

```
p      { color: black; }    /* loses */  
.note { color: blue; }     /* wins – more specific */
```

CSS Resources

- **MDN — CSS** (the go-to reference)
<https://developer.mozilla.org/en-US/docs/Web/CSS>
- **web.dev — Learn CSS** (Google)
<https://web.dev/learn/css>
- **W3C CSS** (the official standard)
<https://www.w3.org/Style/CSS/>
- **Flexbox Froggy** (learn Flexbox by playing)
<https://flexboxfroggy.com>

React State

State is data a component remembers between renders — and changing it **triggers a re-render**. Declare it with the `useState` hook.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

ELI5: Whiteboard vs. Scrap Paper

Picture a **whiteboard** and a piece of **scrap paper**:

- A **re-render wipes the whiteboard clean** and starts again — like a normal variable.
- **Scrap paper** holds the important numbers you want to **remember after** the whiteboard is wiped — that's **state**.

Need a value to survive the next render? Write it on the **scrap paper** (state).

Why State, Not a Variable?

- A normal variable **resets every render**, and changing it **doesn't** trigger a re-render
- **State persists** across renders, and updating it **tells React to re-render**
- Rule of thumb: anything the UI should update in response to → keep it in **state**

Breaking Down `useState`

```
const [count, setCount] = useState(0);
```

- `useState(0)` — call the hook with the **initial value** (`0`)
- it returns an **array of two things**:
 - `count` — the **current value**
 - `setCount` — the **setter** that updates it and triggers a re-render
- `const [count, setCount] = ...` — **array destructuring** names them (convention: `x / setX`)

When Does a Component Re-render?

A component re-renders when:

- its **state** changes (you call a setter like `setCount`), or
- its **props** change (new data from its parent), or
- its **parent** re-renders

React re-runs the component function and updates only what actually changed in the DOM.

Mini Coding Project: React College Schedule Builder

Time to build! Create a React app where a user can **add** classes and **view** their schedule by day and time.

Full details — purpose, requirements, example layouts, and stretch goals — are in the project spec:

```
project_specs/college_schedule_builder.md
```